

AI IN DATA SCIENCE • CHAPTER 2

Lecture 5

Data Collection & Preprocessing

Part 1 — Framing the Problem, Getting & Exploring Real Data

Based on Aurélien Géron — Hands-On Machine Learning, 3rd Ed., Chapter 2 • github.com/ageron/handson-ml3

Today's Roadmap

1

Look at the Big Picture

2

Get the Data — California Housing

3

Create a Test Set (the right way)

4

Visualize to Gain Insight

5

Engineer New Attributes

Dataset

California Housing Prices
1990 census, block-group level
20,640 districts × 10 attributes

Learning Objectives

- ✓ Frame a real ML problem before writing any code — objective, performance measure, assumptions
- ✓ Load a real-world dataset and perform a first structural inspection
- ✓ Explain why the test set must be set aside before exploration (data snooping bias)
- ✓ Create a test set correctly — including stratified sampling
- ✓ Visualize geographical & correlation structure in the data
- ✓ Engineer new attributes by combining existing ones

Look at the Big Picture — Frame the Problem

Before touching any data, a real project starts with framing questions. Skipping this is a common cause of ML projects that work technically but fail to deliver value.

Question	Our answer
Business objective?	Feed predicted prices into a downstream investment-analysis system
Current solution (if any)?	Manual expert estimation — costly, often off by >20%
Supervised, unsupervised, or RL?	Supervised — we have historical prices as labels
Classification or regression?	Regression — a continuous price
Batch or online learning?	Batch — no continuous stream; periodic retraining is fine

Select a Performance Measure

RMSE — Root Mean Square Error

$$\text{RMSE}(X, h) = \sqrt{\left(\frac{1}{m} \sum (h(x^i) - y^i)^2 \right)}$$

The Euclidean (ℓ_2) norm. Sensitive to large errors — a few big mistakes hurt the score a lot.

MAE — Mean Absolute Error

$$\text{MAE}(X, h) = \left(\frac{1}{m} \sum |h(x^i) - y^i| \right)$$

The Manhattan (ℓ_1) norm. More robust to outliers — no single wrong prediction dominates.

Rule of thumb: higher-index norms focus more on large values. Prefer RMSE when large errors are especially undesirable; prefer MAE when the data has many outliers you don't want to dominate the metric.

RMSE vs. MAE: An Outlier's Impact

Error set	Values	RMSE	MAE
A (moderate, no outlier)	[2, 2, 2, 2, 2]	2.00	2.00
B (one large outlier)	[1, 1, 1, 1, 9]	4.06	2.60

A single outlier (9) pushes Errors B's RMSE far above its MAE, while Errors A (no outliers) keeps RMSE and MAE close together.

Preview: the housing dataset's capped price values are exactly this kind of large, systematic outlier — worth remembering once we spot them.

Check the Assumptions

Classic pitfall: you assume the downstream system wants exact prices — but it might only need price CATEGORIES (“cheap / medium / expensive”).

That single assumption would make this a CLASSIFICATION problem, not regression — a completely different approach.

Always confirm with stakeholders before building. For this course: confirmed — the downstream system genuinely wants a continuous price estimate, so regression it is.

Get the Data — Where Real Data Comes From



Relational databases
& data warehouses



Web APIs & third-party
services



CSV / Parquet exports
& shared repositories



Sensors & streaming
logs

In this course: we pull the California Housing data straight from github.com/ageron/data — exactly like pulling from a colleague's bucket or an internal data lake.

Always check: licensing • access rights • provenance (who collected it, how, with what bias?)

Quick Look at the Data Structure

Attribute	Meaning	Type
longitude / latitude	Geographic location	float
housing_median_age	Median age of houses in the district	float
total_rooms / total_bedrooms	Total rooms / bedrooms in the district	float
population / households	District population & household count	float
median_income	Median income (in tens of thousands USD)	float
ocean_proximity	Categorical: distance to ocean	object
median_house_value	Target label to predict	float

Reading `.info()` and `.describe()` Like a Detective

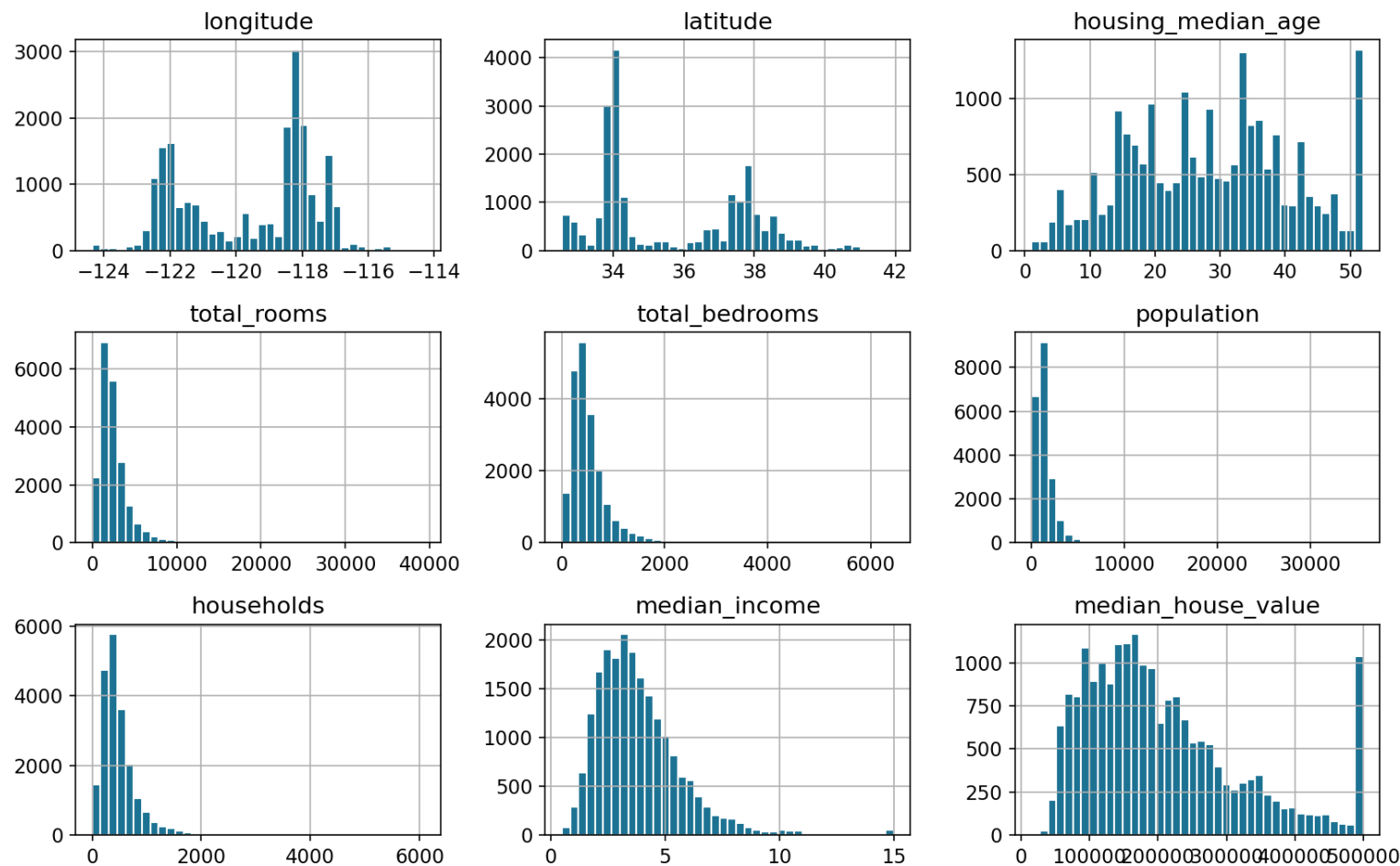
`housing.info()`

- 20,640 rows total
- `total_bedrooms`: only 20,433 non-null → missing data
- `ocean_proximity`: the only non-numeric column

`housing.describe()`

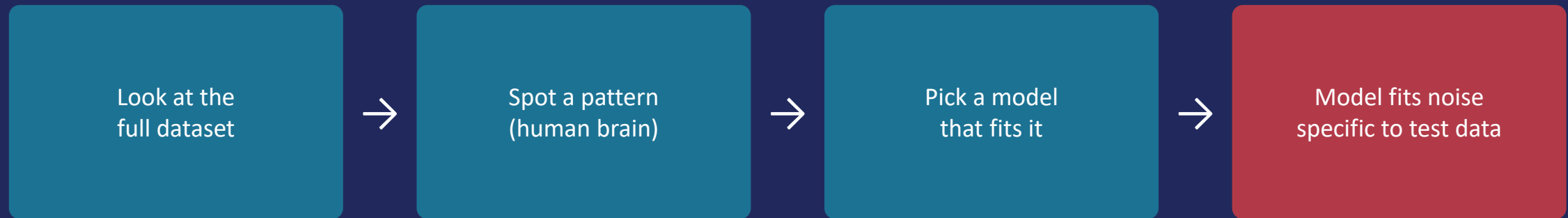
- `total_rooms` max is far beyond the 75th percentile — outlier or data-entry error?
- `housing_median_age` and `median_house_value` both show suspiciously round min/max → capped data
- Attributes live on very different scales → scaling needed later

Every Numeric Attribute — Histograms



- Heavily tail-skewed: total_rooms, population
- Spikes at max value: age & price → capping
- Very different scales across attributes

Set the Test Set Aside — Before You Explore



This is called Data Snooping Bias — an overly optimistic estimate that won't hold up on truly new data.

Rule for this course: split first, explore the training set only, touch the test set exactly once — at the very end.

Step 1 — A Purely Random Split

```
train_test_split(housing, test_size=0.2, random_state=42)
```

80% / 20% split — simple and often fine.

But: median_income is a strong predictor of price. If the random split happens to over- or under-represent certain income brackets, the test set no longer represents the real population.

Solution → Stratified sampling on median_income

Step 2 — Stratified Sampling

Bucket median_income into 5 strata, then sample proportionally from each:

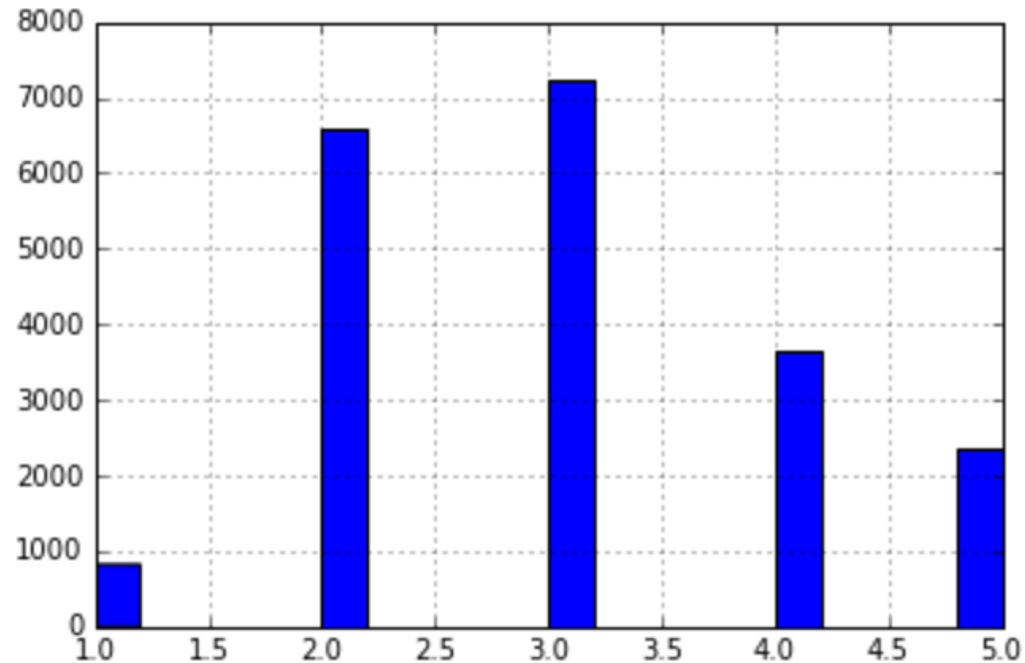
Stratum	Income range	Label
1	0 – 1.5	Low
2	1.5 – 3.0	Below median
3	3.0 – 4.5	Median
4	4.5 – 6.0	Above median
5	6.0 +	High

```
StratifiedShuffleSplit(  
    n_splits=1,  
    test_size=0.2,  
    random_state=42  
)
```

Scikit-Learn draws the split so every stratum keeps the same proportion in train and test as in the full dataset.

Step 2 — Stratified Sampling

Bucket median_income into 5 strata, then sample proportionally from each:



```
StratifiedShuffleSplit(  
    n_splits=1,  
    test_size=0.2,  
    random_state=42  
)
```

Scikit-Learn draws the split so every stratum keeps the same proportion in train and test as in the full dataset.

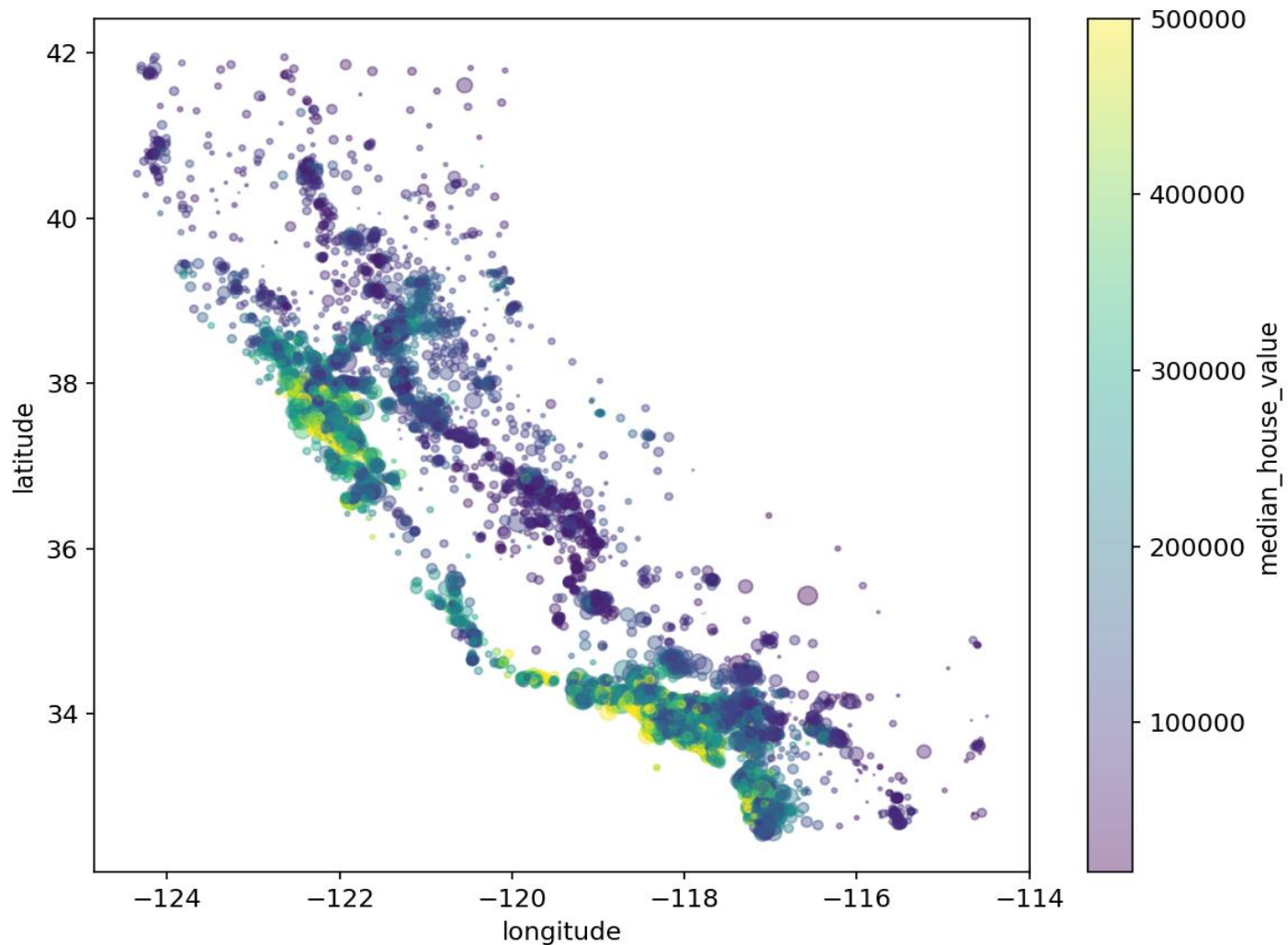
Stratified vs. Random — Quantifying the Bias

Income cat.	Overall %	Stratified %	Stratified err.	Random %	Random err.
1	4.0%	4.0%	+0.37%	4.2%	+6.45%
2	31.9%	31.9%	−0.02%	30.7%	−3.59%
3	35.1%	35.1%	−0.01%	34.5%	−1.53%
4	17.6%	17.6%	+0.03%	18.4%	+4.42%
5	11.4%	11.4%	−0.09%	12.1%	+5.63%

Stratified sampling error stays under $\pm 0.4\%$ in every bucket; random sampling drifts as high as $+6.4\%$.

With the stratum error this small, we drop the helper `income_cat` column — it was only scaffolding for the split — and work only with `strat_train_set` from here on.

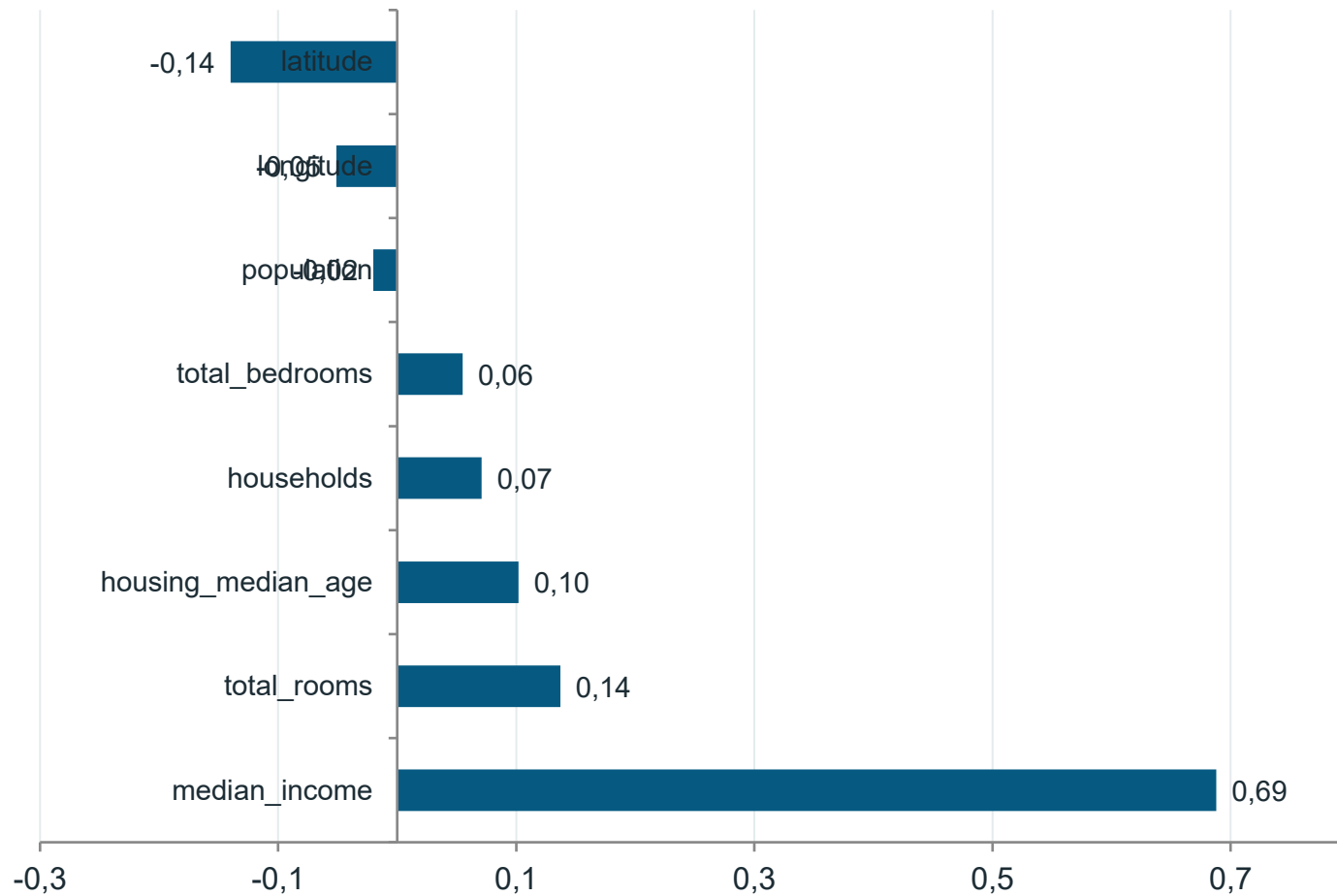
Visualizing Geographical Data



Circle size = population
Color = median house value

- Prices cluster near the coast (Bay Area, LA)
- Dense districts tend to be pricier
- No model trained yet — pure exploratory value
- Lecture 6 will turn this insight into a geographic similarity feature

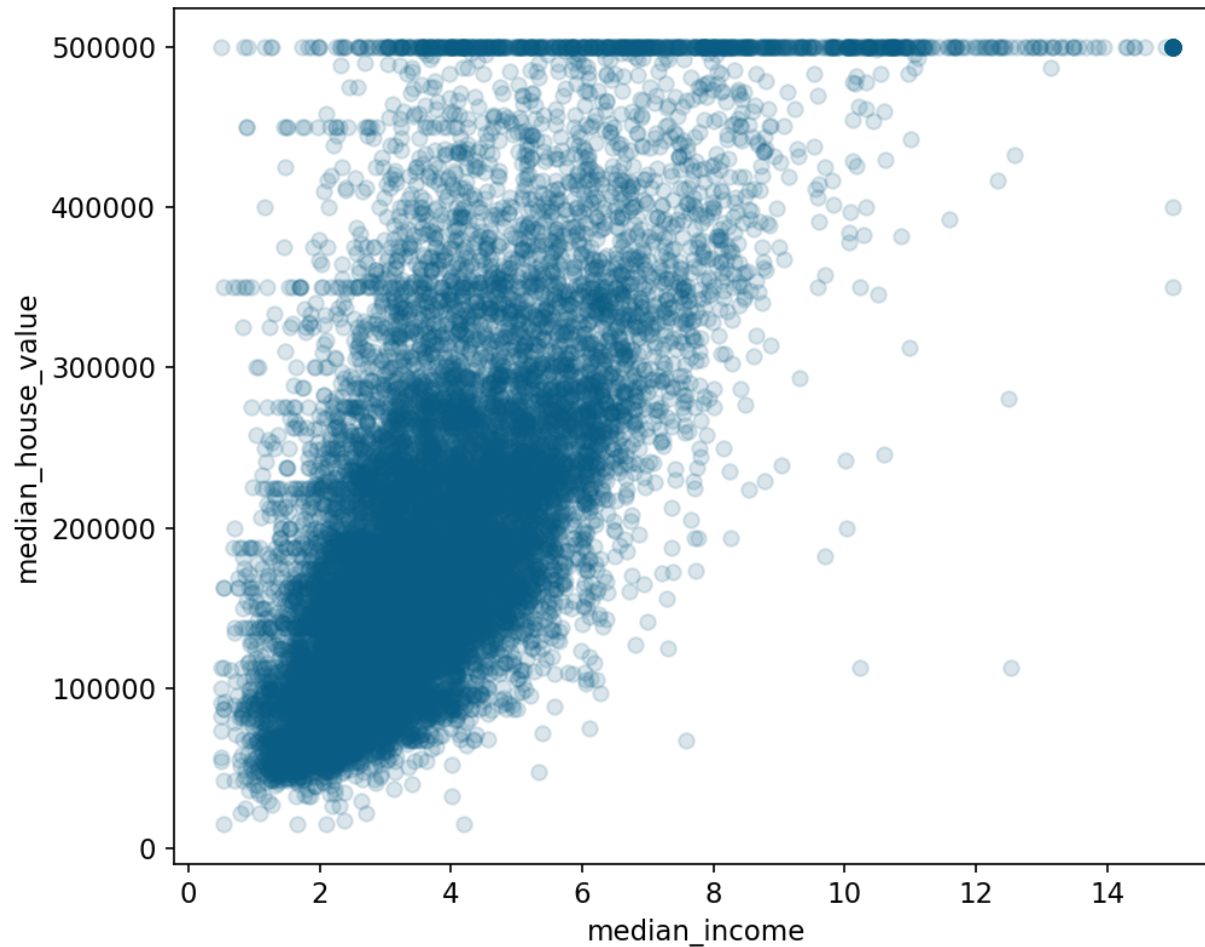
Looking for Correlations with median_house_value



median_income dominates by far (0.69).

Correlation only captures LINEAR relationships — always visualize too (scatter_matrix, scatter plots).

median_income vs. median_house_value



Strong upward trend — our best single predictor.

But look closely: horizontal lines near \$500K, \$450K, \$350K.

These are artifacts of data capping — recall Section 2's RMSE discussion: this is exactly the kind of large systematic error RMSE penalizes heavily. Flag it for data cleaning.

Experimenting with Attribute Combinations

Raw totals aren't very meaningful per se — combine them into ratios:

rooms_per_household `total_rooms / households`

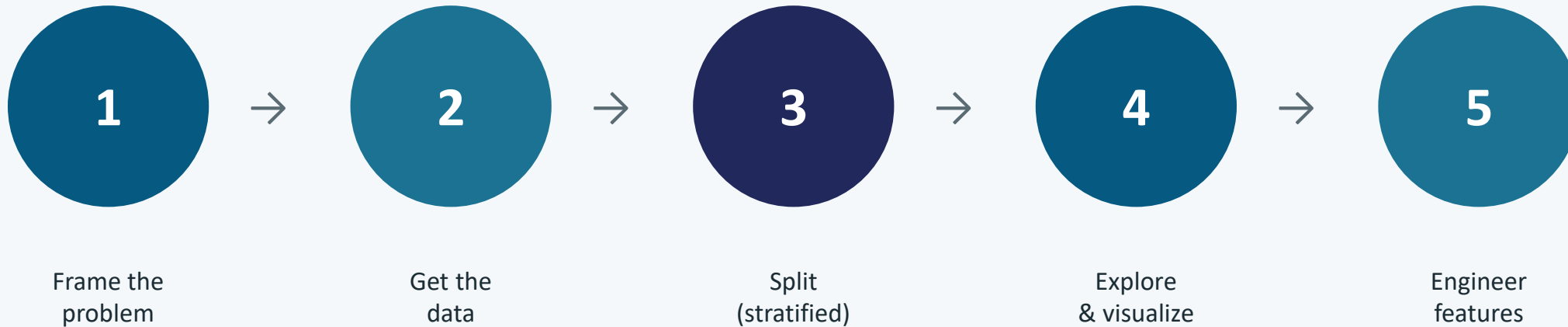
bedrooms_per_room `total_bedrooms / total_rooms`

population_per_household `population / households`

Attribute	Correlation
median_income	0.688
rooms_per_household	0.144
total_rooms	0.137
bedrooms_per_room	-0.256
total_bedrooms	0.055

bedrooms_per_room beats both raw components — the essence of feature engineering.

Today's Pipeline, End to End



Saved to disk: housing_train.csv & housing_test.csv — untouched, ready for Lecture 6.

Next Time & Discussion

Lecture 6 — Data Collection & Preprocessing, Part 2

Data cleaning • categorical encoding • skew correction • geographic similarity features • pipelines • select & train a model

Homework:

- Prepare your virtual environment.